

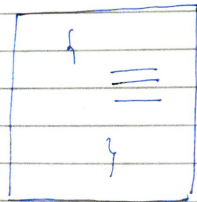
DAY-11.

Function
Method in java;

```
{
    int i = 4;
    int j = 5;
    int sum = i+j;
    System.out.println(sum);
}
```

Repeated code

① Functions — block of code.



perform same task
multiple time in a code
by calling it.

```
public static void main (String[] args) {
    int i = 4;
    int j = 5;
    int result = sum(i,j);
    System.out.println(result);
}
```

arguments

calling a function

```
static int sum (int a, int b) {
    int result = a+b;
    return result;
}
```

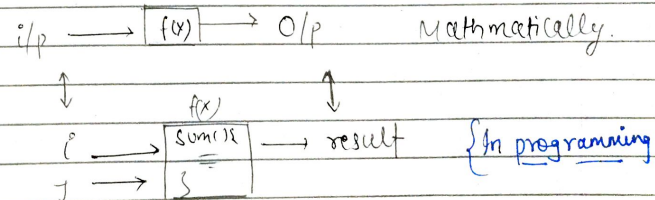
function name

parameters

Returned Type

Returned Type
↓
Datatype that is
returned by
function

Functions



Types of Functions:

- ① No Input, No output
- ② Input, No output
- ③ No input, output
- ④ input, output

Function Overloading:

(called method overriding) in Java.

- Multiple Method in the same class having the same name but different parameters.
- It is concept of compile time polymorphism.

Chaining of Functions:

```
psvm (String[] args) {  
    A();  
    sop("Hi");  
}
```

```
void A() {  
    B();  
    sop("Hello");  
}
```

```
void B() {  
    C();  
}
```

Chaining

Scope of variable;

- ① Global variable / Scope
- ② Local variable / Scope

↳ A local variable is declared inside a Method, Constructor or in block, eg. {

} It can only be used inside that block.

- ① Also any where in class or single file but java does not have true global variable like C/C++;

Recursion;

A function which call itself.

```
main() {  
    A();  
}
```

```
void A() {  
    A();  
}
```

Base case

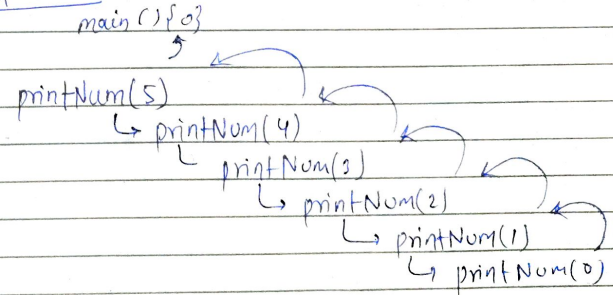
Q. Problem.

→ print Number from 1 to 10 using recursion

```
main() {  
    int n=5;  
    printNum(n);  
}
```

```
void printNum(int n) {  
    if (n==0) return; → (base case)  
  
    printNum(n-1); → Recursive call.  
    System.out.println(n);  
}
```

How it work?



Output;

1
2
3
4
5

Fibonacci Number.

→ 0 1 1 2 3 5 8 13 ...
 $n \rightarrow$ 0th 1st 2nd 3rd 4th 5th

$$Fib(n) = Fib(n-1) + Fib(n-2)$$

```
int fib(n) {
```

```
    if (n == 0 || n == 1) {
```

```
        return 1;
```

```
    }
```

```
    int x = fib(n-1);
```

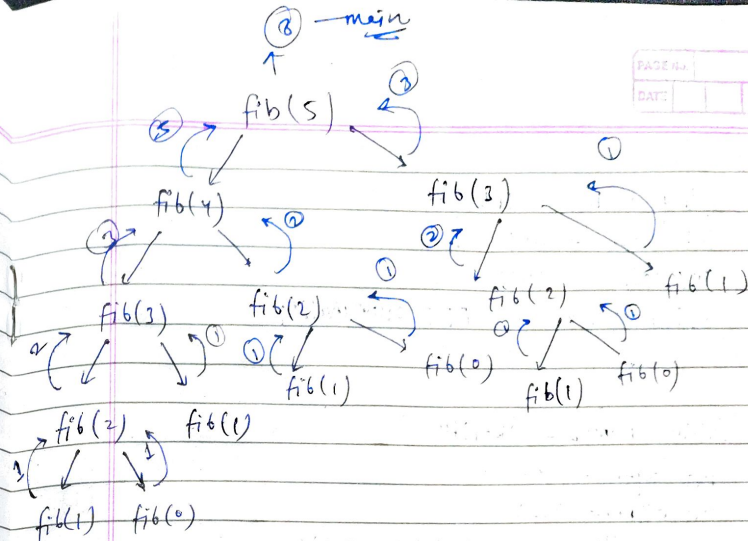
```
    int y = fib(n-2);
```

```
    return x+y;
```

```
}
```

output.

8



→ "Base Case is the case where our recursive call end."